

PRD: Real-Time AI/NLP Financial Risk Engine

Event: S&P Global & Crisil Campus Hackathon 2026
Candidate: Medhansh Poojari, IIIT Ranchi
College email: poojari.2024ug1099@iiitranchi.ac.in
Deadline: Saturday 10 October 2026 (submit by Fri 9 Oct for safety margin)
Team size: 1 (individual submission only)
Status: v1 draft, written 3 Oct 2026

The repo is public, so only the college email is listed here. Do not put personal contact details in any committed file.

1. Summary

Build a pipeline that reads unstructured financial text (news and social posts) and turns it into structured risk signals: **sentiment score**, **event type**, **impact score**. Feed those signals into two downstream applications:

- Module A:** a tactical rebalancer for a mock 15-stock index driven by sentiment.
- Module B:** a strategic stress tester for a synthetic wholesale-banking portfolio (loans, bonds, derivatives, equities) triggered by high-impact events.

One dashboard shows both modules. The demo follows one story: a geopolitical shock appears in the news, the engine flags it, Module A shifts index weights, Module B shows the loss on the bank book.

2. Goals and non-goals

Goals

- A working, explainable engine with at least two ingestion sources and the three required output fields.
- Both modules working end to end on one dashboard, with Module B as the primary showcase (the sponsors are risk and credit specialists).
- Measured results on a labeled test set, compared with a naive baseline.
- A repo, deck and video that follow the organizers' format exactly.
- Candidate can explain every file and design choice in a live technical Q&A.

Non-goals

- Claiming trading profit or market-beating returns. News data is synthetic, so no honest backtest exists.
- Production-grade streaming infrastructure, authentication, or a database.
- Real or confidential client data of any kind.
- Perfect NLP accuracy. A transparent, evaluated baseline beats an opaque model nobody can explain.

3. Hard constraints from the organizers

Requirement	Source	How we satisfy it
Public GitHub repo, named <college>-<name>-hackathon	Guidelines	iiitranchi-medhansh-poojari-hackathon (confirm exact format)

Requirement	Source	How we satisfy it
Repo layout: README.md, requirements.txt, LICENSE (MIT), src/, data/, docs/presentation.pdf, docs/architecture.png	Guidelines	Restructure prototype into src/ (currently a root riskengine/ package)
README must follow the mandatory template (title, candidate info, overview, architecture, dataset, quickstart, results)	Guidelines	Fill the template verbatim, see section 13
Exact run command in README Quickstart	Guidelines	One command, e.g. <code>python -m src.app</code>
Deck of 5-7 slides, PDF in docs/, link in README	Guidelines + case study	Section 14
Demo video on YouTube unlisted , link in README	Guidelines	Test in an incognito window
Video length: conflict. Case study says live demo at most 5 min, guidelines say 10 min recording	Both docs	Record 4-5 min, satisfying both. Ask organizers if unsure
All data used (including synthetic) committed in data/, sources documented, no proprietary data	Guidelines	Section 7
No Google Drive / OneDrive for slides	Guidelines	Slides live in the repo
Incremental commit history preferred	Guidelines	Commit after every task in section 12
Repo free of large binary or data dumps	Guidelines	Small CSV/JSONL only, no model weights
AI use allowed with honesty	Guidelines	Disclose AI assistance in README
Live jury pitch for finalists, technical Q&A	Guidelines	Section 15

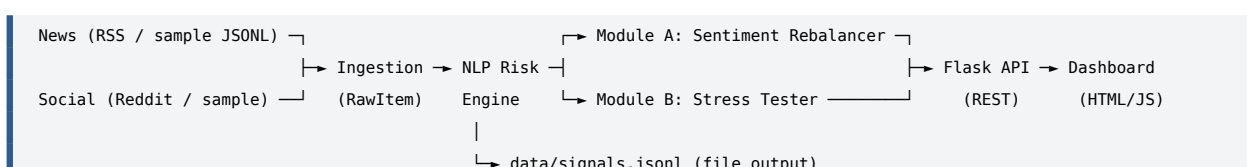
Disqualification risks to avoid: private repo, broken or restricted video/slide links, missing pitch attendance, copied or pre-existing project, any real confidential client data.

Tie-breakers (in order): Domain Understanding, then Presentation & Communication, then earlier submission time. Scoring rubric beyond these is unknown (open question Q3).

4. Users and use cases

- **Jury / evaluator:** needs to clone, run and understand it in minutes.
- **Risk analyst (target persona):** wants early, explainable warning that an event may hurt the book, and by how much.
- **Portfolio manager (target persona):** wants sentiment-aware tilts that stay diversified and bounded.

5. Architecture



Data flow: sources produce `RawItem`; the engine produces `Signal` records (written to `data/signals.jsonl` and served over `/api/signals`); the modules consume signals; the dashboard calls the API.

Stack: Python 3.11+, Flask, standard library for XML/JSON/CSV, plain HTML and JavaScript canvas for charts. `unittest` for tests. Optional: a pretrained finance sentiment model as a stretch goal behind a feature flag.

6. Functional requirements

Priority: **P0** must ship, **P1** should ship, **P2** stretch.

6.1 Risk Engine

ID	Requirement	Priority	Acceptance criteria
E1	Ingest text from at least two sources (news + social)	P0	Both source types appear in <code>signals.jsonl</code>
E2	Live fetchers (RSS, Reddit) with automatic fallback to bundled samples	P1	Network failure never crashes the run; a warning is logged
E3	Sentiment score in [-1, 1] plus label (positive/neutral/negative)	P0	Negated and intensified phrases score as expected in unit tests
E4	Event classification into: Geopolitical, Macroeconomic, Credit Event, Merger/Acquisition, Product Launch, Earnings, Regulatory/Legal, Cyber/Operational, Other	P0	Labeled-set accuracy reported
E5	Impact score 1-10	P0	Always within bounds; catastrophic news scores higher than routine news
E6	Entity linking to tickers (cashtags and company aliases)	P0	Market-wide events have an empty entity list
E7	Output via file and REST API	P0	<code>data/signals.jsonl</code> plus GET <code>/api/signals</code>
E8	Free-text analysis endpoint for live demo	P1	POST <code>/api/analyze</code> returns the three fields

Signal schema (one JSON object per line):

```
{
  "id": "news-010",
  "ts": "2026-10-01T16:10:00Z",
  "source": "wire-sample",
  "source_type": "news",
  "entities": [],
  "sentiment": -0.962,
  "sentiment_label": "negative",
  "event_type": "Geopolitical",
  "event_confidence": 0.62,
  "impact": 9.5,
  "text": "BREAKING: Missile strikes escalate military tensions; ..."
}
```

(Values illustrative; the real file is generated by the pipeline.)

6.2 Module A: Sentiment rebalancer

ID	Requirement	Priority
A1	15 S&P 100 stocks, equal base weight	P0
A2	Positive sentiment raises a stock's weight, negative lowers it	P0
A3	Weights always sum to 1 and stay within 2%-15%	P0
A4	Sentiment fades over time (half-life 90 minutes)	P1

ID	Requirement	Priority
A5	Market-wide events tilt by sector (e.g. geopolitical shock helps energy, hurts tech and banks)	P1
A6	Dashboard chart of weights over time	P0
A7	Compare against a static equal-weight index	P1

6.3 Module B: Stress tester

ID	Requirement	Priority
B1	Synthetic wholesale-banking portfolio mixing loans, bonds, derivatives and equities	P0
B2	Trigger a stress test when impact is above 7 on a negative signal with a mapped event type	P0
B3	Scenario shocks per event type (table in 8.2)	P0
B4	Revalue every position and report before/after totals	P0
B5	Break results down by asset type and show worst positions	P1
B6	One crisis triggers one stress run (collapse repeats within 60 minutes)	P1
B7	Manual scenario run, including the brief's textbook case (equity -10%, rates +200bp)	P1
B8	Dashboard: before/after bars, loss by asset type	P0

6.4 Dashboard

One page with: signal feed with sentiment/impact, source and event mix, Module A weight chart, Module B before/after chart with the triggering headline, and a free-text box that calls `/api/analyze` live. Must work offline from sample data.

7. Data requirements

File	Content	Origin
<code>data/sample_news.jsonl</code>	24 synthetic news headlines across one trading day	Written by us, synthetic (generator: <code>scripts/make_sample_data.py</code>)
<code>data/sample_social.jsonl</code>	29 synthetic social posts with likes/retweets	Written by us, synthetic , mimics X/Reddit style
<code>data/sample_portfolio.csv</code>	25 synthetic positions, about \$1.27B market value	Written by us, synthetic (counterparty names are fictional)
<code>data/labeled_eval.csv</code>	50-100 headlines hand-labeled by the candidate with sentiment and event type	To be created by the candidate
<code>data/signals.jsonl</code>	Engine output	Generated

Rules:

- Document every source and assumption in the README "Dataset Used" section.
- Live sources (RSS, Reddit) are optional extras. Do not commit scraped article text. Commit only headlines you wrote or short metadata, to keep the repo clean and avoid copyright problems.
- If you use a public labeled dataset for evaluation, check its license and cite it. Commit only a small sample if the license allows it.

- The case study mentions "provided sample transaction data" and an open-source dataset list. Neither was received (open question Q2), so the portfolio is synthetic. This is allowed by the guidelines.

8. Method specification

8.1 NLP

- **Sentiment:** finance-tuned word list (weights -3 to +3). Negation within a 3-word window multiplies by -0.74. Intensifiers (e.g. "sharply", "slightly") scale up or down. Emoji and "down 8%" style moves add score. Final score = $\text{total} / \sqrt{\text{total}^2 + 8}$, which keeps it in (-1, 1).
- **Event type:** weighted regular-expression rules per event class; highest total wins; below a minimum score the label is "Other". Confidence is the winner's share of all rule scores.
- **Impact:** $\text{base_severity}(\text{event}) \times (0.6 + 0.4 \times \min(1, |\text{sentiment}| \times 1.5)) + \text{catastrophic-word boost (capped at 2.5, damped } \times 0.3 \text{ if sentiment is positive)}$. Social posts are multiplied by 0.7 and get up to +1 for engagement. Clamped to 1-10. Base severities: Geopolitical 7, Credit Event 7, Macroeconomic 6, Regulatory 5, M&A 5, Cyber 5, Earnings 4, Product Launch 3, Other 2.
- **Why not a neural model:** runs anywhere with no installs, every score is traceable to words, and a live demo cannot fail on a model download. A pretrained finance model is a P2 option, with the word list as fallback.

8.2 Stress scenarios

Event type	Equity	Rates	Spreads	FX	Default prob.	Overrides
Geopolitical	-10%	-50bp	+150bp	-4%	$\times 1.6$	Energy equity +8%, Financials -14%; PD $\times 2.2$ transport, $\times 1.8$ consumer
Credit Event	-12%	-30bp	+250bp	-2%	$\times 2.2$	Financials equity -25%; PD $\times 3.0$ financials, $\times 2.8$ real estate
Macroeconomic	-8%	+200bp	+60bp	-3%	$\times 1.4$	Tech equity -14%, Real Estate -10%; PD $\times 2.0$ real estate
Regulatory/Legal	-4%	0	+30bp	0	$\times 1.1$	Tech equity -8%
Cyber/Operational	-5%	0	+40bp	0	$\times 1.2$	Tech -9%, Financials -7%
Custom (from brief)	-10%	+200bp	0	0	$\times 1.0$	none

Revaluation (P&L per position):

- **Equity:** $MV \times \text{beta} \times \text{equity shock}$ (a sector override replaces this).
- **Bond:** $-MV \times (\text{duration} \times \Delta \text{yield} + \text{spread duration} \times \Delta \text{spread})$. Government bonds ignore spreads.
- **Loan:** $-MV \times \text{spread duration} \times \Delta \text{spread}$, minus exposure $\times \text{LGD} \times (\text{stressed PD} - \text{PD})$.
- **Interest-rate swap:** $DV01 \times \text{rate move in bp}$. **CDS:** $CS01 \times \text{spread move in bp}$. **FX forward:** notional $\times$ direction $\times$ FX shock.

Known simplifications (be ready to defend these): loans are marked to market (banks often hold them at cost); one-factor shocks with no correlations; no convexity; no counterparty credit adjustment; FX shock only touches non-USD forwards. Shock sizes are illustrative and chosen to be plausible, not calibrated to a regulator's scenario.

8.3 Rebalancer

Per-ticker state decays with a 90-minute half-life. Each signal adds $\text{sentiment} \times (\text{impact}/10) \times \text{source weight (news } 1.0, \text{ social } 0.5) \times 1.5$. Signals naming several tickers split their push by

the square root of the count. Market-wide events add the push times a sector beta times 0.6. Weight is proportional to $\text{base} \times \exp(1.2 \times \tanh(\text{state}))$, then floored and capped at 2% and 15% with iterative renormalization.

9. API

Endpoint	Purpose
GET /	Dashboard
GET /api/signals	Signals; filters ticker, event_type, min_impact, limit
POST /api/analyze	Body {text, source_type?, engagement?}, returns sentiment/event/impact
POST /api/refresh	Re-run the pipeline; {live: true} tries live sources
GET /api/rebalance	Weight history and summary
GET /api/stress	Triggered stress runs; optional threshold
POST /api/stress/run	Run a named scenario manually
GET /api/scenarios, GET /api/summary	Scenario definitions, signal statistics

10. Evaluation and success metrics

Targets below are goals to aim for, not results. Report what you actually measure.

Area	Method	Target
Sentiment	3-class accuracy on hand-labeled set vs. naive baseline (always "neutral")	≥ 70% and clearly above baseline
Event type	Accuracy and per-class precision/recall vs. majority-class baseline	≥ 70% overall
Impact	Rank agreement: do hand-picked "major" headlines score above routine ones?	Consistent ordering on a small check set
Rebalancer	Unit tests: weights sum to 1, within bounds, direction correct; turnover reported	All pass
Stress	Hand-calculate 3 positions and match code output; check sign of every sensitivity	Matches to rounding
Reliability	Full demo runs offline from a fresh clone	No errors

The "results" slide should show the measured accuracy table, a before/after stress chart and a weights-over-time chart. Frame results as a demonstration on synthetic data.

11. Current prototype status (honest snapshot)

Location in the sandbox: /home/claude/risk-engine. Not a git repo yet.

Part	State
Lexicon, NLP analysis, ingestion, pipeline	Written and run; produced 53 signals (24 news, 29 social) with sensible output
Rebalancer	Written and run; weights sum to 1, bounds respected

Part	State
Stress tester + portfolio	Written and run; derivative sign errors found and fixed
app.py (Flask API)	Written, not yet run
templates/dashboard.html	Not written
Unit tests, requirements.txt, LICENSE, README, architecture diagram, deck	Not started
Live RSS / Reddit fetch	Untested (no network in the sandbox); fallback exists
Evaluation set and measured metrics	Not started
Layout vs. required src/ structure	Needs restructuring

Known cleanups: remove leftover logic noise in the rebalancer, add edge-case tests, confirm Python version compatibility on your machine.

12. Work plan (7 days)

Date	Work	Done by
Sat 3 Oct	Agree this PRD; set up GitHub repo, MIT license, first commit; install Python and run the prototype	You
Sun 4 Oct	Move code into src/; write requirements.txt; run and fix app.py; unit tests for NLP, rebalancer, stress	Agent, you review
Mon 5 Oct	Build dashboard (weights chart, stress before/after, signal feed, live analyze box)	Agent, you review
Tue 6 Oct	Label 50-100 headlines; run evaluation vs. baseline; tune lexicon if clearly warranted	You label, agent computes metrics
Wed 7 Oct	README (full template), architecture diagram PNG, deck draft	Both
Thu 8 Oct	Finish deck PDF; rehearse; record video; upload as YouTube unlisted; test in incognito	You
Fri 9 Oct	Fresh-clone test on a clean folder; check all links; submit via the official form	You
Sat 10 Oct	Buffer only	

Commit after each task with a clear message so history shows the build.

13. README checklist (mandatory template)

Title line "[Project Title] - S&P Global & Crisil Campus Hackathon", then candidate name, college email, college, demo video link, slide deck link. Sections: 1 Overview / Problem & Approach (2-3 paragraphs), 2 Architecture & Tech Stack (embed diagram), 3 Dataset Used (source, synthetic nature, assumptions), 4 Quickstart (runtime and OS tested, clone/install/run commands), 5 Key Results & Domain Impact. Add an "AI assistance" note.

14. Deck outline (5-7 slides) and demo script

Slides: (1) Title with name and college, (2) Problem and approach, (3) System design with architecture diagram and data flow, (4) Implementation highlights and why each tech choice, (5) Key results with metrics and screenshots vs. naive approach, (6) Domain impact, (7) Limitations and next steps (optional).

Demo script (target 4-5 min): intro 30s; start the app from the README command 30s; walk the story: input headlines, engine output, Module A weights shifting, Module B before/after stress 2-3 min; paste a fresh headline into the live box; results and impact 45s.

15. Live jury Q&A preparation

Prepare short, honest answers to:

- Why a word-list model rather than a transformer, and how would you upgrade it?
- How was impact defined, and how do you know it is reasonable?
- How did you validate the engine? What are its failure cases (sarcasm, negation, ambiguous tickers)?
- Why do shocks have these sizes? How would you calibrate them to regulatory scenarios?
- Why are loans revalued with spreads? What would change using amortized cost?
- How would this scale to true streaming (queue, workers, storage)?
- How do you avoid overreacting to noisy social posts? (source weights, decay, engagement scaling)
- What did AI tools do, and what did you decide yourself?

16. Rules for any coding agent working on this repo

1. Read this PRD first; do not exceed its scope.
2. Work one task at a time and commit after each with a clear message.
3. Run all tests after every change; never claim a test passed without running it.
4. Never invent metrics. Report only numbers you computed, and say how.
5. Use only synthetic or public data. No real client data, no scraped article dumps, no secrets or API keys in the repo.
6. Keep dependencies minimal and listed in `requirements.txt`; the demo must run offline.
7. Keep the repo small: no model weights, no large datasets.
8. Explain non-obvious code in comments; the candidate must be able to defend it.
9. Never push to a remote or change repository visibility without the candidate's instruction.

17. Risks

Risk	Mitigation
Live demo network failure	Offline sample mode is the default path
Weak accuracy on the labeled set	Report honestly, show failure analysis, tune the lexicon on a separate dev split
Can't explain AI-written code	Daily read-through; this PRD's sections 8 and 15 as study material
Video/slides link broken	Incognito test; slides stored in the repo
Time overrun	P2 items are cut first; Module A is cut before Module B
Video length mismatch	Record at most 5 minutes

18. Open questions

1. **Q1:** Exact repo name format and whether the organizers will accept the college prefix as written.
2. **Q2:** Are the organizers' open-source dataset list and "provided sample transaction data" available anywhere?
3. **Q3:** Is there a scoring rubric beyond Domain Understanding and Presentation?
4. **Q4:** Candidate's Python version and OS, for the README "tested on" line.
5. **Q5:** Is the video limit 5 or 10 minutes?

19. Final submission checklist

- ☐ Repo public, MIT LICENSE present, name follows the convention
- ☐ Fresh clone installs and runs with the README commands only
- ☐ `data/` contains every dataset used; sources and assumptions documented
- ☐ `docs/presentation.pdf` (5-7 slides) and `docs/architecture.png` committed
- ☐ YouTube video unlisted, plays in incognito, link in README
- ☐ README follows the template and notes AI assistance
- ☐ Metrics in the deck are the ones actually measured
- ☐ Commit history is incremental
- ☐ Submitted through the official form before the deadline